Examen práctico — Práctica 1 (Propuesta 4): Recursos más recientes con paginación

Duración máxima: 1 h 45 min. Sin Internet ni asistentes de IA.

Dispones de:

- Tu implementación de la práctica 1.
- Las 16 pruebas completas de la práctica.
- Las 10 pruebas del grupo **X** correspondientes a este examen.
- Seminario de C++ en https://www.dlsi.ua.es/asignaturas/eda/seminario-c++-parte1.pdf
- Copia del manual de referencia de C++ en https://www.dlsi.ua.es/asignaturas/eda/cpp

Ejecuta `./run_tests.sh X` para ejecutar las pruebas del examen y ver tu calificación.

Ampliación: listar los recursos más recientes por páginas

Se quiere consultar el catálogo como un ranking de novedades: los recursos ordenados del más reciente al más antiguo, y poder pedirlos **por páginas** (saltando los primeros). El listado se muestra en el mismo formato que el fichero de entrada, para poder copiarlo y volver a cargarlo.

Todos los métodos de esta ampliación son **añadidos**: no puedes cambiar ninguna firma de la práctica (los grupos A, B y C deben seguir pasando). Recuerda declarar los métodos nuevos en `Resource.h` y `Catalog.h`.

1. Método `Resource::toFileLine`

```
string toFileLine() const;
```

- Devuelve la línea que representa al recurso en el **formato de fichero** de la práctica (Sección 5 del enunciado de la práctica):

```
<id>|<year>|<title>|<tag1>,<tag2>,<tag3>
```

- Las etiquetas van separadas por comas **sin espacios**.
- Si el recurso no tiene etiquetas, el último campo queda **vacío** (la línea termina en `|`).
- **No** añade salto de línea al final.
- Es la operación inversa de la línea que lee `loadFromFile`: cargar un fichero y volcarlo debe reproducirlo, salvo por la normalización de las etiquetas.

Ejemplos:

```
bk01|1999|Clean Code|craft,software
bk02|2020|Empty Tags|
```

2. Método `Catalog::latest`

```
vector<Resource> latest(int k, int skip) const;
```

- Considera todos los recursos del catálogo ordenados por **año descendente** y, a igualdad de año, por **`id` ascendente**.
- Descarta los `skip` primeros de ese orden y devuelve los `k` siguientes.
- Si `k <= 0`: devuelve un vector vacío.
- Si `skip < 0`: se trata como `0`.
- Si `skip >= size()`: devuelve un vector vacío.
- Si tras el salto quedan menos de `k` recursos, devuelve los que haya.
- El catálogo **no se modifica** (el método es `const`).

Casos que debes tener en cuenta: catálogo vacío, empate de años resuelto por `id`, `skip` justo en el límite (`skip == size()`) y `k` mayor que el número total de recursos.

3. Comando `LATEST <k> <skip>`

- Imprime `COUNT <n>` seguido de `n` líneas con los recursos, en el orden indicado, usando el formato de `toFileLine` (**no** el de `operator<<`).
- `n` puede ser menor que `k`.
- Si falta algún argumento o si `k` o `skip` no son enteros válidos: `INVALIDCMD`.

Ejemplo:

```
LOAD data/sample_ok.txt
OK 6
LATEST 2 0
COUNT 2
bk06|2020|Modern C++|cpp,programming
bk03|2015|Design Patterns|design,oop,patterns
LATEST 2 1
COUNT 2
bk03|2015|Design Patterns|design,oop,patterns
bk05|2011|Structure and Interpretation|cs,lisp,programming
LATEST 5 4
COUNT 2
bk01|1999|Clean Code|craft,software
bk04|1994|The Pragmatic Programmer|craft,software
LATEST 3 10
COUNT 0
LATEST 3
INVALIDCMD
```

(El orden completo del catálogo de ejemplo es `bk06` (2020), `bk03` (2015), `bk05` (2011), `bk02` (2008), `bk01` (1999) y `bk04` (1994). La última consulta válida pide 5 recursos saltando 4, pero sólo quedan 2.)

Evaluación

- Nota = número de pruebas pasadas.
- El código debe compilar sin errores ni warnings.
- No se puede modificar la carpeta `tests/`.